



Smart Contract Security: Vulnerabilities & OpenZeppelin Mitigations

MDT915 Week 4 — Session 7 — Workshop 6

Student ID: 8718179

**Student
Name:** Seif Sid Ali Maloufi

Date: 15 March 2026

Student Details

Student Name	Seif Sid Ali Maloufi
Student ID	8718179
Date	15 March 2026
Module	MDT915 Blockchain Implementation

Assignment Instructions

This workshop requires you to critically analyse four Smart Contract Weakness Classifications (SWC) and their corresponding OpenZeppelin mitigations. You are expected to:

1. Explain the vulnerability in your own words, including how it can be exploited on-chain
2. Describe the OpenZeppelin library or pattern used to mitigate the vulnerability
3. Compare and contrast at least two of the four vulnerabilities — discuss their similarities, differences, severity, and the trade-offs of their respective mitigations
4. Support your discussion with Solidity code snippets, examples, or references where appropriate

Question 1: SWC-100 — Function Default Visibility

1.1 Vulnerability Explanation

SWC-100 identifies functions that omit an explicit visibility specifier. In Solidity versions prior to 0.5.0, functions without explicit visibility defaulted to **public** visibility. This means that even if a developer intended a function to be internal or private, it would inadvertently become callable by any external caller.

On-chain exploitation: An attacker can call functions that were intended to be internal or restricted. For example, a contract might have an internal `updateBalance()` function that modifies critical state. If it defaults to public, the attacker could call it directly and manipulate the contract's state, potentially draining funds or corrupting data.

```
// VULNERABLE: Default visibility = public (pre-0.5.0)
contract Vulnerable {
    uint256 public balance;
    function updateBalance(uint256 _newBalance) { // Implicitly public
        balance = _newBalance;
    }
}
```

1.2 OpenZeppelin Mitigation

OpenZeppelin contracts enforce **explicit visibility** for all functions. Solidity 0.5.0+ requires explicit visibility, but the OpenZeppelin library goes further by using strict pragma and audited patterns.

```
// FIXED: Explicit visibility
contract Secure {
    uint256 private _balance;
    function updateBalance(uint256 newBalance) internal {
        _balance = newBalance;
    }
    function getBalance() external view returns (uint256) {
        return _balance;
    }
}
```

Mitigation: Use `pragma solidity ^0.8.20` to ensure the compiler rejects implicit visibility.

Question 2: SWC-105 — Unprotected Ether Withdrawal

2.1 Vulnerability Explanation

SWC-105 occurs when a contract allows withdrawal of funds without proper access control. Any address can call the withdrawal function and

drain the contract's ETH balance.

On-chain exploitation: An attacker monitors the contract for accumulated ETH. When the balance is sufficient, they call `withdraw()` with their address as the recipient. All funds are sent to the attacker.

```
// VULNERABLE: No access control
contract Vulnerable {
    function withdraw() external {
        payable(msg.sender).transfer(address(this).balance);
    }
}
```

2.2 OpenZeppelin Mitigation

OpenZeppelin's **Ownable** contract provides the `onlyOwner` modifier. Restrict withdrawal to the contract owner. For complex scenarios, use **AccessControl** with role-based permissions.

```
import "@openzeppelin/contracts/access/Ownable.sol";
contract Secure is Ownable {
    function withdraw() external onlyOwner {
        (bool success, ) = owner().call{value: address(this).balance}();
        require(success, "Transfer failed");
    }
}
```

Question 3: SWC-106 — Unprotected SELFDESTRUCT Instruction

3.1 Vulnerability Explanation

SWC-106 is a critical vulnerability where the `selfdestruct` opcode can be invoked without access control. The `selfdestruct` instruction permanently destroys the contract and sends all remaining ETH to a specified address. If unprotected, anyone can call it and destroy the contract, stealing all funds.

On-chain exploitation: An attacker calls `destroy(attackerAddress)`. The contract is immediately destroyed, all ETH is sent to the attacker, and the contract is permanently removed from the blockchain. **Real-**

world example: The Parity Wallet incident (2017) — a user accidentally became "owner" of a library contract and called `kill()`, freezing ~\$300M in ETH.

```
// VULNERABLE: Anyone can destroy
contract Vulnerable {
    function destroy(address payable _to) external {
        selfdestruct(_to); // Attacker uses their own address!
    }
}
```

3.2 OpenZeppelin Mitigation

Use **Ownable** with the `onlyOwner` modifier. For high-value contracts, consider **AccessControl** with a multisig (e.g., Gnosis Safe).

```
import "@openzeppelin/contracts/access/Ownable.sol";
contract Secure is Ownable {
    function destroy(address payable _to) external onlyOwner {
        require(_to != address(0), "Invalid recipient");
        selfdestruct(_to);
    }
}
```

Question 4: SWC-107 — Reentrancy

4.1 Vulnerability Explanation

SWC-107 (Reentrancy) occurs when a contract makes an external call before updating its internal state. A malicious contract can call back into the victim contract before the first call completes, allowing multiple withdrawals or state manipulations before balances are updated.

On-chain exploitation: The attacker deploys a contract with a `receive()` function that calls the victim's `withdraw()` again. The victim sends ETH before updating the balance, so the attacker's balance is still non-zero. The attacker can drain the contract repeatedly. **Famous example:** The DAO hack (2016) — ~\$60M stolen due to reentrancy.

```
// VULNERABLE: State update after external call
contract Vulnerable {
    mapping(address => uint256) public balances;
```

```
function withdraw() external {
    uint256 amount = balances[msg.sender];
    (bool ok,) = msg.sender.call{value: amount}(""); // External
    require(ok);
    balances[msg.sender] = 0; // Too late – attacker re-entered
}
}
```

4.2 OpenZeppelin Mitigation

OpenZeppelin's **ReentrancyGuard** provides a `nonReentrant` modifier. Apply the **Checks-Effects-Interactions (CEI)** pattern: update state before external calls.

```
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
contract Secure is ReentrancyGuard {
    mapping(address => uint256) public balances;
    function withdraw() external nonReentrant {
        uint256 amount = balances[msg.sender];
        balances[msg.sender] = 0; // Update state first (CEI pattern)
        (bool ok,) = msg.sender.call{value: amount}("");
        require(ok);
    }
}
```

Question 5: Comparison and Critical Analysis

5.1 SWC-105 vs SWC-106: Similarities and Differences

Aspect	SWC-105	SWC-106
Root cause	Missing access control	Missing access control
CWE	CWE-284: Improper Access Control	CWE-284: Improper Access Control
Severity	High	Critical

Impact	Funds stolen	Funds stolen + contract destroyed
Reversibility	Contract remains functional	Contract permanently destroyed
Mitigation	Ownable / AccessControl	Ownable / AccessControl

Similarities: Both stem from improper access control. Both allow unauthorized parties to extract or redirect ETH. The same OpenZeppelin mitigations (Ownable, AccessControl) apply.

Differences: SWC-106 is more severe because the contract is permanently destroyed. SWC-105 leaves the contract intact; SWC-106 breaks any system depending on the contract (e.g., Parity). SWC-106 has no recovery path.

Trade-offs: Using Ownable centralises control in the owner — a single point of failure. AccessControl with multisig (e.g., Gnosis Safe) is more secure but adds complexity and gas cost.

5.2 SWC-107 vs SWC-106: Severity and Mitigation

Trade-offs

Aspect	SWC-107	SWC-106
Root cause	Incorrect state ordering	Missing access control
Exploit pattern	Recursive call	Direct call
Mitigation	ReentrancyGuard, CEI	Ownable, AccessControl
Gas cost	ReentrancyGuard adds ~2.4k gas	Ownable adds minimal overhead

Severity: SWC-107 can drain funds repeatedly in a single transaction; SWC-106 destroys the contract and steals all funds. Both are critical in production.

Mitigation trade-offs: ReentrancyGuard uses a storage slot and adds a check on every protected call — a small gas cost for strong protection.

Ownable is lightweight but requires careful key management.

References

- SWC Registry: <https://swcregistry.io/>
- OpenZeppelin Contracts:
<https://github.com/OpenZeppelin/openzeppelin-contracts>
- SWC-100: <https://swcregistry.io/docs/SWC-100>
- SWC-105: <https://swcregistry.io/docs/SWC-105>
- SWC-106: <https://swcregistry.io/docs/SWC-106>
- SWC-107: <https://swcregistry.io/docs/SWC-107>